



FIREBRAND

LYCEUM EDUCATION GROUP

Testing, Debugging and Validation in Power Apps

Power Platform Pathway | Session 7

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, K9, K11

Skills

S1, **S4, S6, S7**, S10,
S11, S12, S16

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Testing, Debugging and Validation in Power Apps

Lesson Objectives

Apply

- Apply testing techniques within apps

Analyse

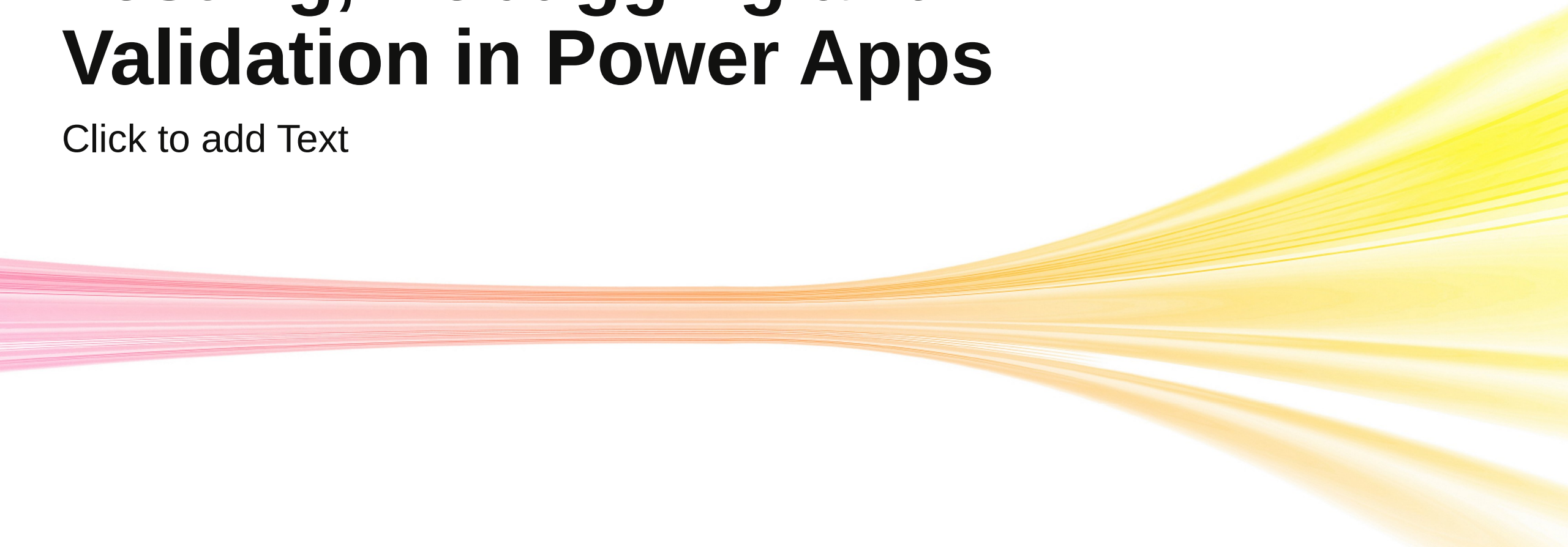
- Analyse behaviour to identify issues

Evaluate

- Create structured test scenarios

Testing, Debugging and Validation in Power Apps

Click to add Text

A decorative graphic consisting of several overlapping, wavy lines that originate from the left side of the slide and extend towards the right. The lines are colored in a gradient of pink, orange, and yellow, creating a sense of motion and energy.

Testing Within the SDLC

Testing identifies defects early, before an app reaches business users

- Shifting testing left in the Software Development Life Cycle (SDLC) catches defects early, preventing critical runtime bugs from reaching business production environments
- **The SDLC Rule:** The earlier a defect is caught, the cheaper it is to fix before reaching corporate business users

UI ERRORS

- Absolute positioning
- Overlapping layers
- Responsive scaling

LOGIC ERRORS

- Order of operations
- Division-by-zero
- Sandbox variables

DATA ERRORS

- Delegation blocks
- Broken schemas
- Blank payloads

User Interface (UI) Errors

Core Vulnerabilities

Hardcoded absolute XY coordinates breaking mobile responsive scaling

Visual Symptoms

Controls overlapping structural containers, text clipping, and hidden action buttons

Logic & Formula Errors

Core Vulnerabilities

Arithmetic order-of-operations failures and unhandled blank states

Visual Symptoms

Application components crashing silently or showing wrong calculations without throwing a syntax error

Data & Integration Issues

Core Vulnerabilities

Ignoring blue underline delegation warnings and field schema type mismatches

Visual Symptoms

Galleries dropping records past index 500, patch failures, and disconnected data variables

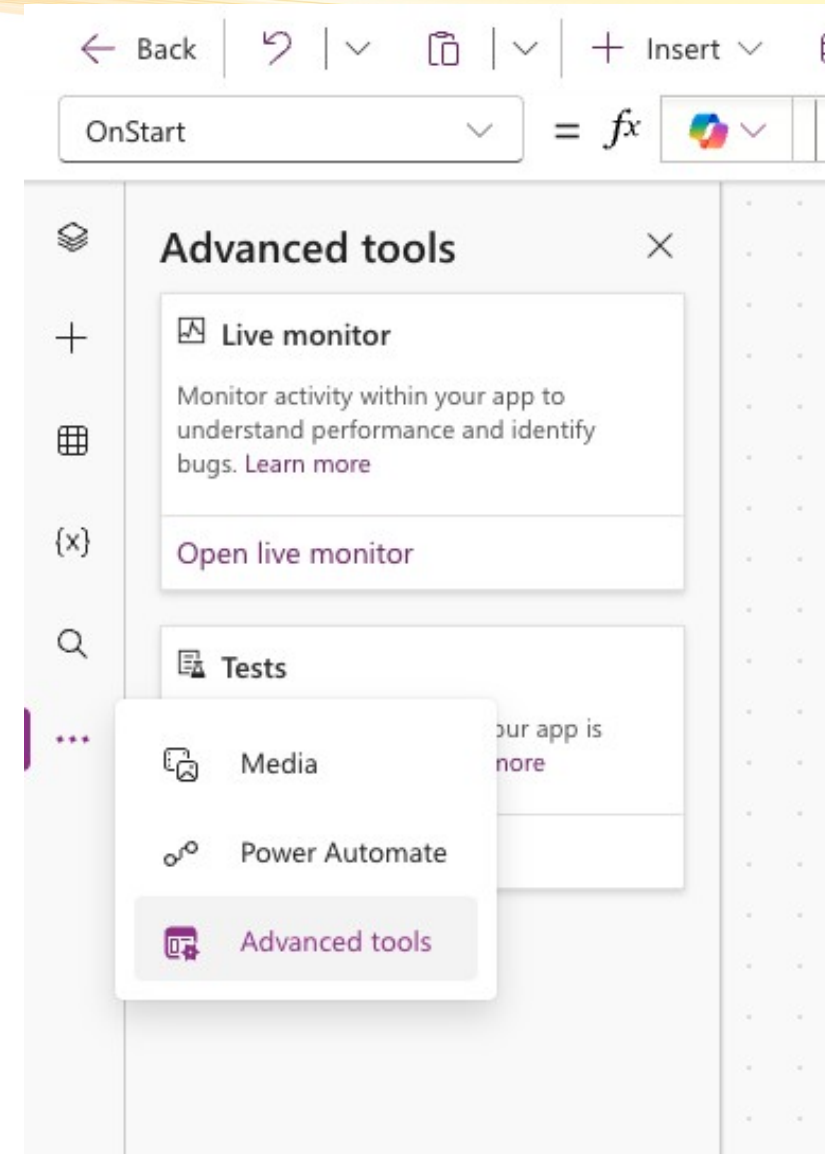
Using the Monitor Tool

The Power Apps Monitor acts as a live flight recorder

It allows low-code engineers to view deep, real-time background streams of data payloads, network actions, and property calculations without altering the app canvas

HOW TO LAUNCH:

Studio Sidebar → Advanced Tools → Open Monitor



Network Parameters

- **What it tracks:** Every external API call, database connection handshake, and security token header
- **Debugging Metric:** Isolates slow network trips, high-volume server payloads, and broken HTTP response codes (e.g., 404 Not Found or 500 Server Error)

Control State Tracing

- **What it tracks:** The exact sequential execution path of formulas whenever a user triggers a control event
- **Debugging Metric:** Displays visual click streams, input string entries, and screen navigation transition times side-by-side with your code properties

Error Isolation & Call Stack

- **What it tracks:** Runtime errors, failed database updates (Patch), and unhandled logic exceptions
- **Debugging Metric:** Pinpoints the exact name of the broken control, the specific line of code that failed, and the source error code thrown by the cloud database

Automated Verification via Test Studio



Building Automated Regressions:

- **The Interface Path:** Open the **Advanced Tools** icon on the left panel menu layout and select **Open tests** to launch the dedicated Power Apps Test Studio environment
- **Recording User Workflows:** Use the visual recording suite to track and save user interactions, including click pathways, field entries, and screen navigation triggers
- **Automated Execution:** Run saved interaction suites instantly on demand to confirm that new code drops do not cause regression defects in existing app modules.

Enforcing Business Logic with Assertions:

- An **Assertion** is an engineering expression that tests whether an app state matches your precise success criteria.
- If an expression evaluates to true, the test passes successfully. If it returns false, the suite halts execution and logs a structural failure

// Test Studio Assert Formula Example

```
Assert(  
    Value(lbl_CalculatedTotal.Text) = 5000,  
    "Validation Failure: Calculated math drift detected  
inside order total container."  
)
```

Real-Time State Diagnostics

Inspecting In-Memory Assets:

- **The In-Memory Blindspot:** Logic errors frequently happen because data fields change silently inside memory collections without throwing hard user interface crashes
- **The Variables Interface:** Open the **Variables** tab on the left panel tree view layout to track down variable locations, active data values, and modification logs
- **The Collections Interface:** Open the **Collections** view panel to inspect live, tabular data arrays currently stored in the device's temporary memory stack

Common Memory Audit Checks:

- **Type Mismatch Audits:** Check that text strings are not accidentally being forced into strict numeric data columns
- **Null Record Checks:** Confirm that a server query returned actual data rows rather than an empty, blank record container
- **Field Mapping Verification:** Verify that schema columns have not been dropped or renamed across collection modification threads

Defensive Error Interception

Graceful Error Handling:

- Unhandled system errors cause input elements to go blank, which breaks data integrity and compromises user trust
- The **IfError()** function intercepts runtime processing failures before they impact the layout, allowing you to run clean fallback logic

Formula Failures Handling Matrix

// UNPROTECTED APPROACH: App crashes silently if connection drops

```
Patch(InventoryTable, TargetRecord, { StockCount: NewValue })
```

// PROTECTED APPROACH: Error is caught and a clean fallback is triggered

```
IfError(  
    Patch(InventoryTable, TargetRecord, { StockCount: NewValue }),  
    Notify(  
        "System Warning: Data sync interrupted. Retrying connection  
link...",  
        NotificationType.Warning  
    )  
)
```

Production Engineering Best Practice



Always wrap transactional commands (such as `Patch()`, `SubmitForm()`, and external API Custom Connector calls) inside an `OnError()` wrapper to maintain a reliable user experience under unstable network conditions!

Designing Test Cases

- **Typical journey:** a normal user completing the main task
- **Edge case:** boundary values (e.g. an empty gallery)
- **Invalid input:** text entered where a number is expected

TYPICAL JOURNEY

- Happy Path
- Verified via:
 - Test Studio Automation

EDGE CASE

- Boundaries
- Verified via:
 - IsBlank()
 - IsEmpty()

INVALID INPUT

- Type Safety
- Verified via:
 - IsNumeric()
 - IsBlank()

The Typical Journey (The Happy Path)



- **Definition:** A standard user completes the primary transactional workflow without error
- **Power Apps Target:** Automated interaction paths recorded in the **Test Studio Suite**
- **Success Metric:** Form submits smoothly with no notification errors

The Edge Case (Boundary Conditions)



- **Definition:** System behavior when data variables hit maximum or minimum limits
- **Power Apps Target:** Handling empty gallery grids or missing user records
- **Validation Formula:**
`If(IsEmpty(colOrders), Reset(gal_Catalog))`

The Invalid Input (Type Validation)

- **Definition:** High-risk entries where the wrong data type is typed into a field
- **Power Apps Target:** Text characters entered where a strict numeric integer is required
- **Validation Formula:**

`If(!IsNumeric(txt_PriceInput.Text), Notify("Error: Number required."))`

Spot the Formula Error

Flawed Formula (Logical & Operational Risks)

// CRITICAL BUG: Order of operations error drops calculations by 1

`Sum(colOrders, Amount) / CountRows(colOrders) - 1`

// RUNTIME CRASH: If collection is empty, app divides by zero and fails

`Sum(colOrders, Amount) / CountRows(colOrders)`

Correct Formula (Defensive Engineering Path)

// SECURE SOLUTION: Intercepts zero counts and protects calculations safely

```
If(
    CountRows(colOrders) > 0,
    Sum(colOrders, Amount) / CountRows(colOrders),
    0
)
```

Structured Debugging Lab



ACTIVITY



Objective: Diagnose and fix an app containing multiple issue types

Tasks / Requirements:

- Identify the type of issue present (UI, logic/formula, or data)
- Trace the source of the problem through the app
- Correct formulas, controls or data connections as appropriate

Key Point: Evidence a structured troubleshooting approach: identification, analysis, correction, validation

Extension Activity: Write two additional test cases covering edge cases for the fixed screen



ACTIVITY



Test Design Workshop

Creating Structured Test Frameworks for Canvas Apps

THE 3 TEST SCENARIOS

- *Typical Journeys (Happy Path)*: Verifying standard, error-free user navigation workflows
- *Edge Cases (Boundaries & Nulls)*: Forcing blank search strings and empty collection variables
- *Invalid Inputs (Type Safety)*: Keying letters into numeric form text boxes

WORKSHOP TASKS:

1. *Define expected outcomes*: Map out precise expected visual behaviours
2. *Document test data variables*: Create a fixed ledger of test parameters
3. *Set success criteria*: Establish clear pass or fail validation markers
4. *Review test plan coverage*: Verify all functional screen formulas are audited

Validation Activity



ACTIVITY



Objective: Execute test scripts against a working app and validate canvas application functionality

Tasks / Requirements:

- **Run cases:** Execute test cases against live application
- **Record outcomes:** Log live actual results step-by-step
- **Compare variance:** Bench actual metrics against expected criteria
- **Identify failures:** Highlight, flag, and document operational inconsistencies

Key Point: Always retest functionality following code corrections

Extension Activity: Automate validation routines using native Test Studio suites



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Evaluation Discussion



The Business Case for Testing



**Application
Reliability**



**User Confidence
and Trust**



**Database Quality
and Integrity**

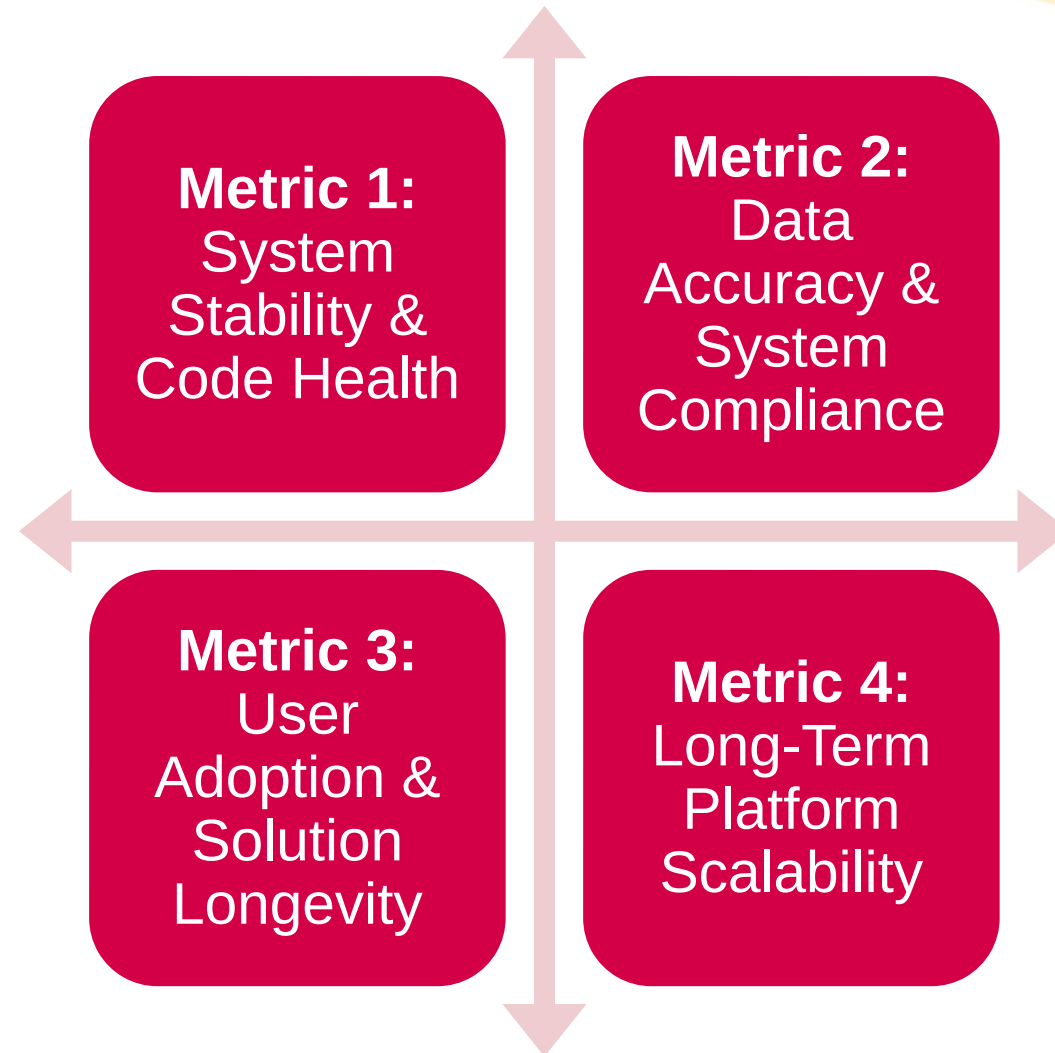


**Business
Efficiency**

The Cost of Defect Leakage



Business Impact of Testing



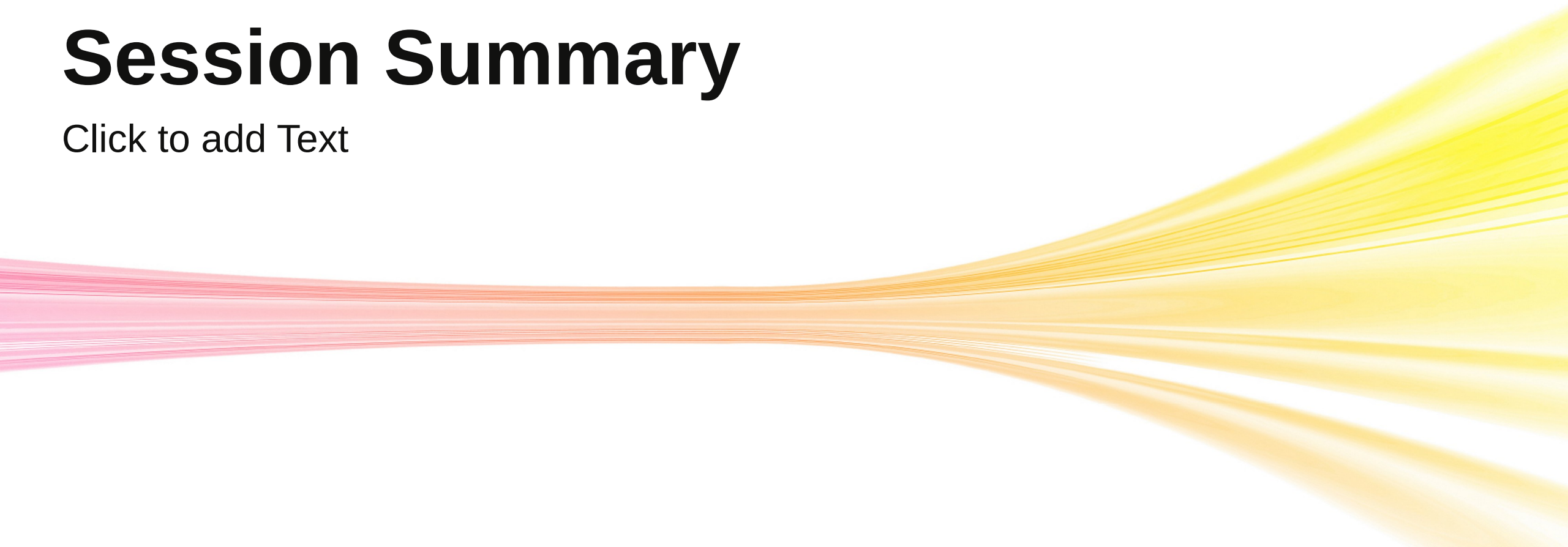
Stretch and Challenge

The Mission: demonstrate the ability to evaluate and improve testing coverage, ensuring a more robust and reliable application

- **Task 1:** Review an incomplete test plan
- **Task 2:** Identify gaps in test coverage, focusing heavily on validation rules and key user journeys
- **Task 3:** Add missing test scenarios
- **Task 4:** Justify how additional tests improve confidence in the solution
- **Task 5:** Review whether all critical user workflows and field validation rules are thoroughly tested

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Testing, Debugging and Validation in Power Apps

Lesson Objectives

Apply

- Apply testing techniques within apps

Analyse

- Analyse behaviour to identify issues

Evaluate

- Create structured test scenarios

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, K9, K11

Skills

S1, **S4, S6, S7**, S10,
S11, S12, S16

Behaviours

B2, B3

